1 Preparacion para el TP2

1.1 Estructuras elegidas

1. Usuarios — AVL/ABB para la busqueda normal y HEAP para obtener el ranking por millas (Esto abarcaría crearUsuario, RegistrarUsuario y RegistrarUsuarioConReferidor). El arbol del usuario va a estar compuesto por nodos, dichos nodo contendrán el ID del usuario, su categoría, su saldoDisponible (Es decir las millas disponibles) y tambien las totales, esto para aprovechar tambien y obtener instantamentamente su categoría.

2. Transacciones — ArrayList(Transaccion): (Abarca los procs CanjearMillas,TranferirMillas, AcumularMillas y tambien ParesTransferidoresExclusivos!)

3. Promociones — DiccionarioDigital (TRIE): como clave tiene el nombre de la promocion, y como valor al objeto Promocion. (Abarca los procs IniciarPromocion,IniciarPromocionConTope y FinalizarPromocion)

4. ElMasAcumulador: Heap(HandleUsuario) ElMasAcumulador es una estructura Heap que contiene el handle del usuario, es decir, una referencia abstracta de los usuarios es mucho mas fácil acceder al Objeto usuario en tiempo O(1). A su vez la clase ElMasAcumulador implementa la interfaz comparator que se encarga de chequear que para todo objeto del heap este primero siempre el que mas millas acumuló.

5. elMasCanjeador: Heap(HandleUsuario) Misma idea con el mas canjeador, tiene un handler para acceder a la referencia del usuario en O(1) e implementa el mismo comparator pero con un CompareTo pensado para la

6. rankingMillas: Heap(HandleUsuarios), esto entiendo que costaría N log U porque obtener los N usuarios es O(1) y reordenar el heap sería N veces log u, es decir O(n log U), la diferencia entre este y los dos anteriores es que aca hay que reordenar el heap y eso cuesta O(N log U). Ademas, esta clase del heap utiliza otro comparator, que compara justamente por el que tiene mas millas. Creo que igual aca hay que tener un cuidado extra. Estamos SACANDO los elementos del Heap, si luego no los vuelvo a insertar, no se rompe el invariante pero se puede romper el sistema. Esto lo podemos consultar.

   Falta todavia alguna estructura o algo para ver como va a ser ParesTransferidoresExclusivos...

1.2 Objetos que vamos a usar

1. Usuario: tiene un ID, una categoria, las millas disponibles y las millas totales

2. Promocion: tiene un nombre, un factor y millas restantes. Al crearlo se debe especificar si es con o sin tope

1.3 Procedimientos / Funciones de AirMiles

1. Canjear Millas, Transferir Millas: tardan O(log U) porque buscar al usuario en el arbol cuesta eso, y asignarle las millas es O(1)

2. Acumular Millas: tiene complejidad O(log U + $|L|$) porque la busqueda del usuario es log U y la busqueda de la promocion es $|L|$ ya que su estructura es un TRIE

3.

1.4 Objetos auxiliares

1. Clase Transaccion:
   Enum TipoDeTransaccion (Canjeo, transferencia, referido)
   int UsuarioOrigen
   int UsuarioDestino
   int distancia
   int millas
   str NombrePromo
   float factorPromo

   Hay que definir tambien aca los métodos del objeto transaccion (tener solo una clase con atributos, lo vuelve una clase anémica. No tiene mucho sentido. Otra opcion sino sería usar una estructura como un struct o algo asi aunque no la vimos.)

2. Objeto Promocion
   str NombrePromocion
   float FactorPromocion
   bool TieneTope
   float MillasRestantes

   (Creo que los métodos se definen con un override porque realmente quien los implementa es la clase Airmiles que pisa los metodos del objeto promoción)
   metodos:
   IniciarPromocion
   IniciarPromocionConTope
   FinalizarPromocion